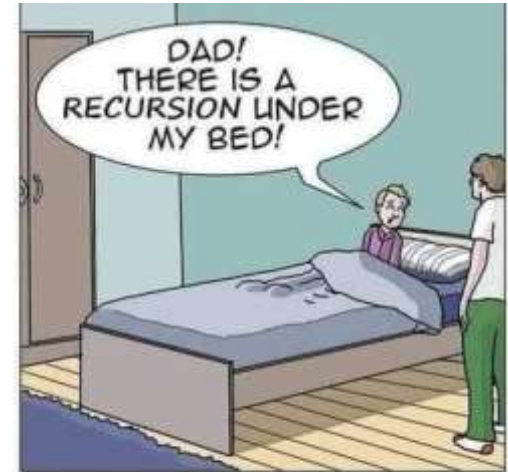


Recursion

What is Recursion?

- Act of a method calling itself.
- Known as a recursive method.
- Will keep calling itself if there is no condition to stop.
- A way of problem solving by breaking down a broader problem into small parts.
- Mostly used as an alternative of loop especially when using loop can be quite difficult.
- Follows bottom to top then top to bottom approach (or can be considered vise versa as per convenience).



What is Recursion? (Cont.)

- Pros : You may have to write less code.
- Cons : Might have to deal with quite a few errors!

Loop versus Recursion

Loop	Recursion
Not much difference in terms of use. Both can be used to solve repetitive problems.	
Constructs like for, while used.	Method calls itself and consists of a base case and recursive case.
Loop condition given to stop loop execution at a point.	Stops executing when base case is reached.
May not be easy to write for some complex problems.	May be easier to write complex problems and have more clarity.
Is more memory efficient.	Possibility of using more memory and resources due to stack overflow error.

Components of Recursive Method

Two components:

- Base Case
 - Termination condition of the recursion process.
- Recursive Case
 - Other cases other than the base case where the method keeps calling itself.

Structure of a Recursive Method

```
static return_type recursive_method(datatype parameter){  
    if (base case condition is satisfied){  
        return final_value;  
    }  
    else{                                     //recursive case  
        recursive_method(argument);  
    }  
}
```

Can you tell what could be the issue if there was no base case?

Method Call from Another Method

We all have seen in the previous “Method” lecture where we can call a method from another method.

Ex: Finding the sum of a simple series $1 + 2 + 3$

- method1 will return the sum of the first two numbers to method2.
- method2 will add the returned value with 3 and return it to the method2 call in the main method where the returned result is printed.

```
public class MethodExample {  
    public static void main(String[] args) {  
        System.out.println(method2(3));  
    }  
    static int method1(int a, int b){  
        int sum1 = a + b;  
        return sum1;  
    }  
    static int method2(int b){  
        int sum2 = method1(1, 2) + b;  
        return sum2;  
    }  
}
```

```
Welcome to DrJava. Working directory is  
C:\Users\Desktop
```

Method Calling with Recursion

A recursive method works in a similar way. Except that the method will keep calling itself to keep adding the numbers and give the desired result.

And also, make sure the code works for sum of n natural numbers.

Ex: $1 + 2 + 3 + 4 + \dots n$

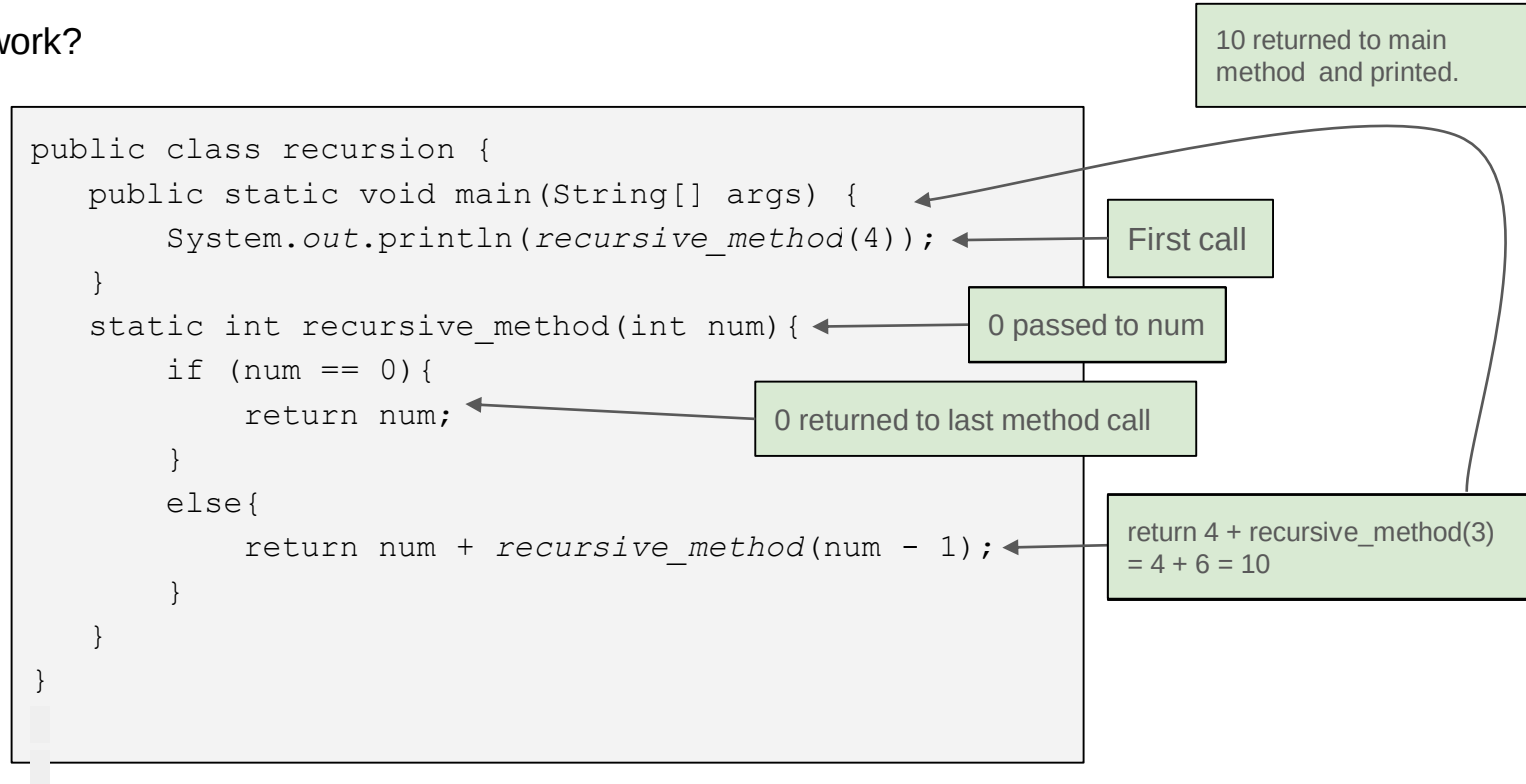
Change the number 4 in the method call to any other number and see if it works. You can also take n as user input.

```
public class recursion {  
    public static void main(String[] args) {  
        System.out.println(recursive_method(4));  
    }  
    static int recursive_method(int num){  
        if (num == 0){                //base case  
            return num;  
        }  
        else{  
            //recursive case  
            return num + recursive_method(num - 1);  
        }  
    }  
}
```

```
Welcome to DrJava. Working directory is  
C:\Users\Desktop  
10
```

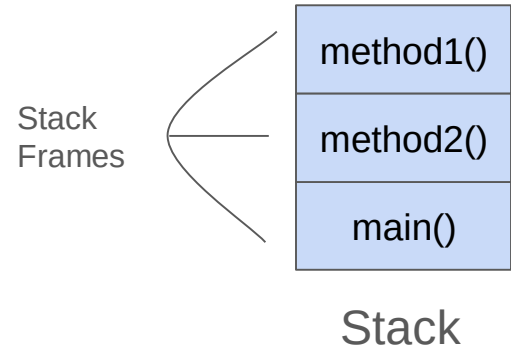

Method Calling with Recursion (Cont.)

How does this work?



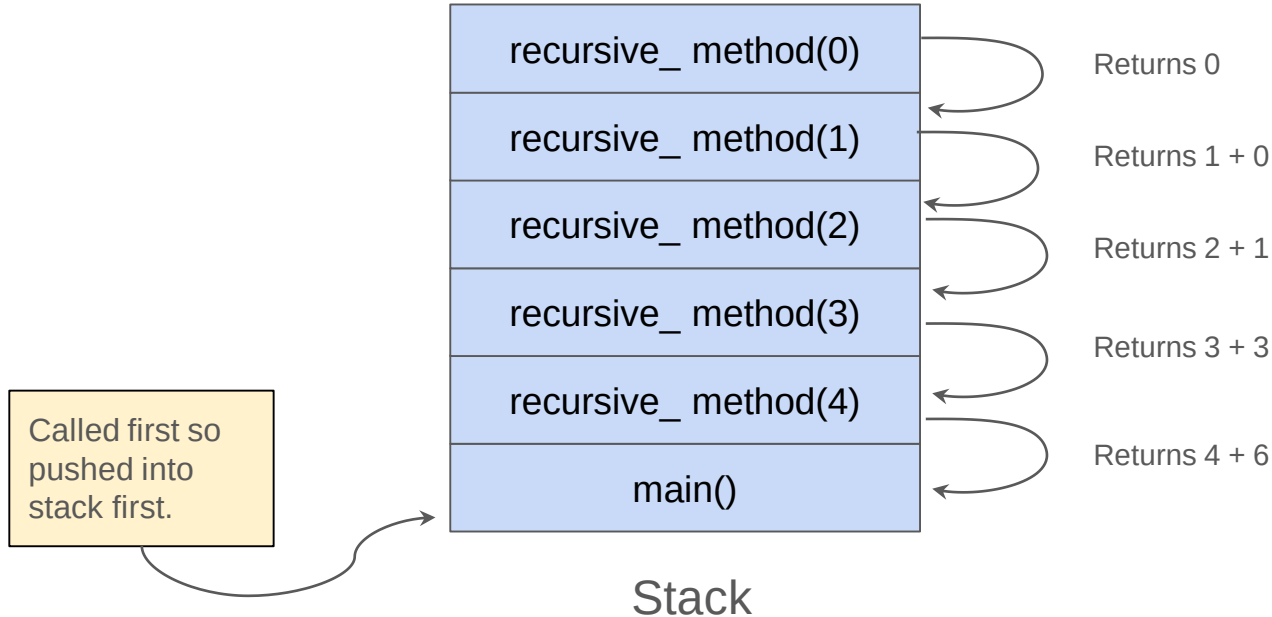
Method Call Stack

- Used to keep track of method calls in a data structure called **Stack**.
- Each method, when called, gets allocated into the stack frame.
- Each stack frame consists of the information about the called method like its local variables, arguments passed, returned value etc.
- A call stack example shown on the right following the example given in Slide 7.



Method Call Stack (Cont.)

- For recursive method, every method call is also allocated a stack frame.
- A call stack is shown for the previous example of recursion [Slide 9].

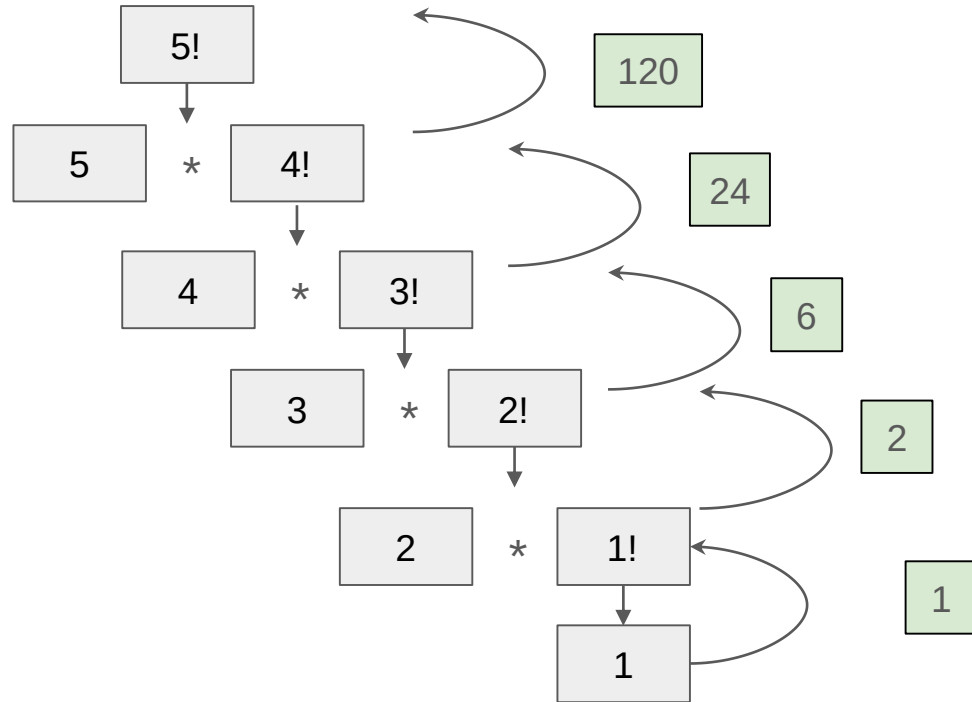


Factorial using Loop

```
public class factorial {  
    public static void main(String[] args)  
    {  
        int fact = 1;  
        int number = 5;  
        for (int i = 1; i<=5; i++){  
            fact *= i;  
        }  
        System.out.println(fact);  
    }  
}
```

Factorial Problem Breakdown

- Factorial of a number can be broken down into smaller parts.

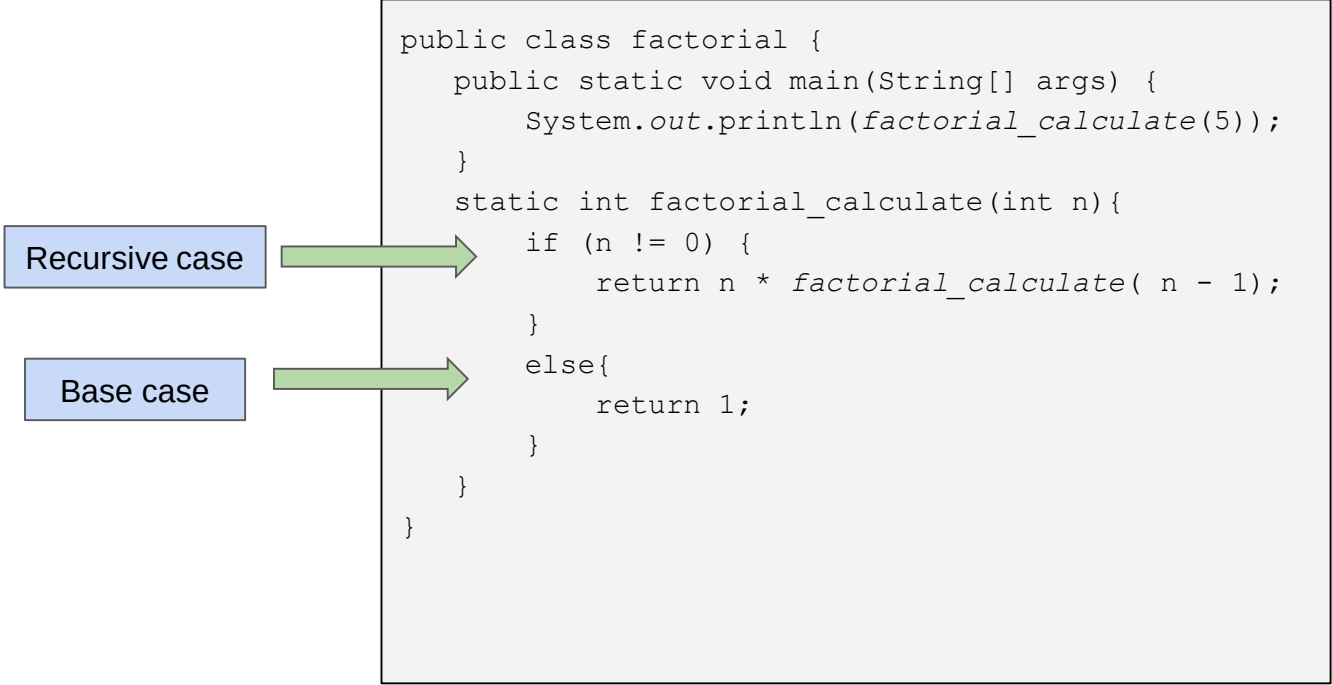


Factorial using Recursion

Recursive case

Base case

```
public class factorial {  
    public static void main(String[] args) {  
        System.out.println(factorial_calculate(5));  
    }  
    static int factorial_calculate(int n){  
        if (n != 0) {  
            return n * factorial_calculate( n - 1);  
        }  
        else{  
            return 1;  
        }  
    }  
}
```

The diagram illustrates the recursive calculation of a factorial using Java code. On the left, two blue boxes are present: 'Recursive case' and 'Base case'. Green arrows point from these boxes to the corresponding code blocks in the Java snippet. The 'Recursive case' arrow points to the 'if (n != 0)' block, which shows the function calling itself with 'n - 1'. The 'Base case' arrow points to the 'else' block, which returns the value 1, representing the termination condition of the recursion.

More Examples of Recursion

- Finding the minimum or maximum value in an array
- Fibonacci series
- Sorting algorithms, ex: selection sort, insertion sort etc.

THANK YOU!